Dreamer from Pixels: A Reproducible Cartpole Swingup Pilot

Daniel Yang
Carnegie Mellon University
CMU 11-785
DDaniel-code/dreamer-pixels

Abstract

This project studies whether the core Dreamer V1 pipeline can be reproduced from image observations in the DeepMind Control Suite. We fixed a concrete PyTorch implementation, audited its source and runtime assumptions, and ran a bounded single-seed pilot on `cartpole_swingup`. The formal run used 64 x 64 RGB pixels, action repeat 2, 50,000 policy decisions (100,000 underlying environment steps), and 4,900 optimizer updates. A ten-update gate verified finite, nonzero updates to the world model, actor, and critic. On the same ten evaluation seeds, the final policy obtained mean return 181.701 (sample standard deviation 27.571), compared with 70.684 for the pre-update policy and 55.540 for an auxiliary random policy. These results are initial evidence that the end-to-end path ran; they are not a full paper reproduction, a multi-seed result, or a claim of reaching the paper's performance.

1 Overview and Context

Motivation and objective. Dreamer is a useful reinforcement-learning-from-pixels baseline because it must learn a visual representation, a temporal dynamics model, and a control policy rather than relying on simulator state. Our objective is to reproduce the central Dreamer V1 training path under an explicit observation and evaluation contract, then use the validated baseline for extensions. The first task is `cartpole_swingup`; `walker_walk` and `ball_in_cup_catch` are reserved for later work. The primary metric is episodic return and its learning curve, not classification accuracy.

Research target. The target algorithm is Dreamer V1 from Hafner et al. [1]. The selected executable source for this pilot is the PyTorch repository by Julius Frost at commit `ccea6ae4a397a94c328891bd78e81d52dd156cb6`, with its `rlpyt` submodule at `f04f23db1eb7b5915d88401fca67869968a07a37`. This is treated as a V1 implementation, not silently upgraded to Dreamer V2 or V3. The original Google Research TensorFlow repository and Danijar's TensorFlow 2 repository remain historical and audit references. The selected source is used because it provided a runnable actor–critic path in the available Kaggle PyTorch environment; its MIT license and upstream attribution were retained.

2 Related Work and Background

Dreamer learns a latent recurrent state-space model (RSSM) from image sequences and improves the policy by imagining trajectories in latent space. An encoder maps pixels to an embedding. The recurrent state contains deterministic memory and stochastic state variables; a posterior uses the current observation while a prior predicts the next stochastic state from history and action. The model reconstructs observations and predicts rewards and continuation. The actor receives gradients through

33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada.

Table 1: Effective configuration of the completed formal pilot.

Task / observation	cartpole_swingup, camera 0, 64 x 64 RGB
Training seed	0
Decision / environment budget	50,000 / 100,000
Action repeat	2
Prefill	1,000 decisions / 2,000 environment steps
Train schedule	every 1,000 decisions, 100 updates
Batch size / length	16 / 50
Replay capacity	60,000 decisions
Precision / anomaly detection	32-bit / disabled
Evaluation seeds	1000–1009

imagined dynamics, while the critic estimates imagined returns. Replay therefore stores temporal sequences, not independent image/action pairs.

The implementation audit found that paper-level descriptions and repository defaults are not a single interchangeable configuration. The paper defines the algorithm and evaluation motivation, while the code fixes operational choices such as replay layout, batch scheduling, checkpoint format, and logging. We record those choices explicitly rather than inferring unspecified paper values. The project repository adds documentation, configuration records, tests, export tooling, and compatibility notes; it does not relicense or silently replace upstream learning code.

3 Methodology

3.1 Model and data path

For a real sequence, the encoder produces an embedding from the 64 x 64 RGB observation. The RSSM uses a posterior $q_\theta(s_t \mid h_t, o_t)$ and a prior $p_\theta(s_t \mid h_t)$, where h_t summarizes previous latent state and action. The world-model objective combines image reconstruction, reward prediction, continuation prediction, and a KL regularizer between posterior and prior. Starting from posterior states, the actor samples actions and propagates imagined states through the learned dynamics; the critic regresses to imagined value targets. This is the main reason the project checks that gradients are finite and that all three parameter groups actually change.

The environment is DeepMind Control Suite `cartpole_swingup`. The agent sees pixels only; no simulator position, velocity, or other low-dimensional truth is passed to the model. Camera 0 renders 64 x 64 RGB observations. One policy decision repeats its action for two environment transitions, so the reported environment-step axis is twice the decision-step axis. Each episode is capped at 500 decisions, or 1,000 underlying environment steps. The replay capacity is 60,000 decision transitions, which is 120,000 underlying environment transitions under this contract.

3.2 Evaluation protocol

The mandatory baseline is the selected Dreamer V1 path. Random action is an auxiliary control, not a replacement baseline. Random, pre-update formal initial, and final Dreamer policies use the same ten environment seeds, observation contract, action repeat, and 500-decision episode limit. The initial checkpoint is the same formal training model after prefill and before its first optimizer update; it is not a separately initialized policy. We report the mean and sample standard deviation of the ten episodic returns. A return learning curve is shown only for the run's recorded training-return diagnostics; the final policy comparison is kept separate from the training curve.

4 Baseline Selection and Experimental Iteration

The first implementation route used the historical TensorFlow code path. It exposed Python 3.13 and Keras/API incompatibilities, including missing MuJoCo dependencies and failures in old optimizer/module interfaces. We stopped that route rather than changing the algorithm through an unbounded sequence of compatibility patches. The PyTorch route was then checked against the actual environment: a CPU child process imported `dm_control`, loaded `cartpole_swingup`, reset,

Table 2: Pass/fail evidence for the pilot.

Check	Status
CPU import, reset, ten random steps	Passed
CUDA forward/backward and visible GPU	Passed
EGL pixel rendering	Passed
Ten real Dreamer updates	Passed
World model, actor, critic parameter changes	Passed
Fresh-process checkpoint evaluation	Passed
100,000 environment-step single-seed run	Passed
Full paper reproduction / multi-seed replication	Not completed
Full optimizer, replay, and environment-state resume	Not tested

Table 3: Ten-episode evaluation on the same seeds (1000–1009).

Policy	Mean return	Sample std.
Random control	55.540	42.597
Formal initial policy	70.684	3.150
Final Dreamer pilot	181.701	27.571

and executed ten random actions; a fresh GPU process passed CUDA matrix/gradient checks and headless EGL pixel rendering. The previously validated `dm-control -no-deps` installation route was reused because ordinary dependency resolution attempted to build `labmaze`.

An important configuration error was corrected before the formal run. An intermediate setup used `train_every=1` and `train_steps=8`, which would produce roughly 80 times more updates per interaction than the selected upstream schedule. The formal run restored `train_every=1000` and `train_steps=100`. A second correction enlarged the replay capacity from an approximately 1,060-transition debugging buffer to an independently configured 60,000-transition buffer. These changes affect compute load and temporal data coverage, so they are recorded as iteration evidence rather than hidden implementation details.

5 Baseline Reproduction Evidence

The ten-update debug gate completed a real optimizer step path with finite diagnostics. Maximum absolute parameter changes were 0.011696 for the world model, 0.000710 for the actor, and 0.000771 for the critic. A checkpoint was written and loaded in a fresh process for evaluation. This validates a short update and evaluation path, but not full optimizer/replay continuation.

The independent formal run completed 50,000 decision steps and 100,000 environment steps on one visible Tesla T4. The first row of the saved progress file is the prefill boundary: iteration 999, 1,000 decisions, and zero optimizer updates. The final row reports iteration 49,999, 50,000 decisions, and 4,900 cumulative updates. The training subprocess reported 4,438.8 seconds of training time; the final fixed-seed evaluation reported 636.2 seconds. The downloaded archive passed a complete ZIP integrity check and contains the configuration, raw metrics, checkpoint, logs, evaluation CSV, and source manifests.

6 Results and Analysis

Table 3 reports the fixed-seed comparison. The final policy has a higher mean return than both controls in this bounded run, and its parameters differ from the initial formal checkpoint. That observation is useful as initial baseline evidence, but it does not establish statistical significance or paper-level reproduction: there is one training seed, one task, and a limited interaction budget. The random control also has substantial across-seed variability.

Failure and limitation analysis. The local macOS pixel path was not treated as a training result because its MuJoCo rendering backend was unavailable under the bounded check. The original TensorFlow path also failed under the available modern Python/Keras environment. On Kaggle,

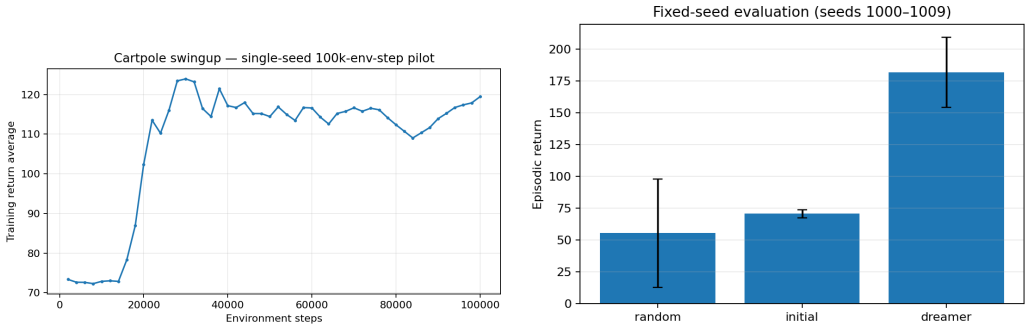


Figure 1: Left: recorded training-return diagnostic versus the decision-step axis, with the corresponding environment-step scale equal to twice the decision steps. Right: fixed-seed evaluation comparison. These are derived from the downloaded raw artifacts; no performance values were filled in by hand.

the standard `dm-control` install attempted a `labmaze` build; the no-dependency route and EGL configuration resolved the runtime without changing the learning objective. These failures identify environment portability risks, not evidence that the algorithm itself failed. The current checkpoint was verified for loading and evaluation, but full continuation of optimizer state, replay contents, and environment state was not tested. Finally, the final comparison is a single-seed pilot and must not be described as complete reproduction.

6.1 Sensitivity and extensions

No dropout/stale-frame ablation or world-model prediction figure is claimed here. They were intentionally paused until the baseline path, checkpoint export, and evaluation contract were reliable. The next experimental comparison should use the same evaluation seeds for at least two additional independent training seeds, followed by a pre-registered multi-seed summary. Only after that should we test the planned stale-frame condition or extend to `walker_walk` and `ball_in_cup_catch`.

7 Discussion and Future Directions

The main midterm result is methodological: the project now has a fixed V1 source, a documented pixel/action-repeat contract, a real actor–critic update path, and downloadable raw evidence. The Kaggle pilot is sufficient to show that a small GPU budget can support a meaningful baseline check. It is not a substitute for a controlled university-scale replication, because Kaggle sessions, package images, storage persistence, and quota policy are external variables. The school environment still needs confirmation of SSH and compute-node access, scheduler use, CUDA/driver versions, Python or container policy, and project/scratch storage.

Before claiming a reproduction, we will (1) repeat the same configuration with two additional training seeds, (2) compare the resulting learning curves and final returns under the unchanged evaluation protocol, and (3) test complete checkpoint continuation including optimizer and replay state. We will then add the two reserved tasks and use the same distinction between decision steps and underlying environment steps. A research extension will be selected only after the standard Dreamer V1 baseline is reproducible.

8 Conclusion

We implemented and measured a bounded Dreamer V1 pixel-control pilot on `cartpole_swingup`. The ten-update gate and 100,000-environment-step run both produced real updates to the world model, actor, and critic, and the final checkpoint was evaluated in a fresh process. The initial results are encouraging but limited: one training seed and one task are not enough to claim paper reproduction. The repository and report therefore preserve both the evidence and the unresolved replication work.

Team Contributions

Daniel Yang completed project framing, source audit, implementation integration, Kaggle execution, artifact extraction, analysis, and report preparation. Kangping Liu is the listed TA mentor; no mentor code was supplied. The current project materials contain no additional group-member roster, so no unverified names are added.

GitHub

Project repository: `https://github.com/DDaniel-code/dreamer-pixels`. The repository records the fixed upstream commit, configuration differences, raw-artifact manifest, and the notebook used for the completed pilot. Large experiment artifacts remain outside Git and are linked by local evidence paths.

References

[1] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ronald Ivanovic, and others. Dream to Control: Learning Behaviors by Latent Imagination. arXiv:1912.01603, 2019. `https://arxiv.org/abs/1912.01603`.

[2] Yuval Tassa, Saran Tunyasuvunakool, Ivan Muldal, and others. DeepMind Control Suite. arXiv:1801.00690, 2018. `https://github.com/google-deepmind/dm_control`.

[3] Julius Frost. Dreamer PyTorch implementation, commit `ccea6ae4a397a94c328891bd78e81d52dd156cb6`. MIT-licensed source. `https://github.com/juliusfrost/dreamer-pytorch`.

[4] Google Research. Original Dreamer V1 reference implementation. `https://github.com/google-research/dreamer`.